

Tether

Product Specification — v0.1

IDEATION PHASE

PRE-DEVELOPMENT

MAY 2026

1 — VISION

Music has always been an intimate act. The moment you hand someone one of your earbuds, you're inviting them into something personal. Modern streaming killed the physical cord — but nothing replaced the feeling.

Tether brings that moment back. When you open your profile, you're on stage — friends can see what you're listening to and drop in, synchronizing to your exact playback position. It's not a collaborative playlist. It's presence.

2 — CORE CONCEPT

The Metaphor

Sharing a wired earbud. When you "Match Their Vibes," you borrow one of a friend's earbuds — you hear exactly what they hear, at the exact moment they hear it. When they skip a track, you skip. When they pause, you pause. You're in it together.

Two Modes of Connection

The product supports two emotionally distinct use cases, and the design must honor both:

MODE	DESCRIPTION	TO NE
Active	Listening together while on FaceTime or a call — the friend is showing you something, a favorite part of a song, a new album drop.	Energetic, shared, communicative
Silent / Ambient	Quietly connecting to a friend — maybe an old one, maybe late at night. No chat, no pressure. Just presence.	Intimate, low-stakes, almost invisible

The UI should never force the active experience on someone who wants the silent one.

3 — FEATURE SET

3.1 Profile as a Live Stage

- Every user has a profile that shows, in real time, what they're currently listening to — track, artist, album art.
- Actively listening users display a subtle live indicator.
- Any friend can visit the profile and see the live playback state.
- Listening activity is only visible to approved friends (see Privacy Model).

3.2 Match Their Vibes (Core Sync Feature)

- From a friend's profile or friend list, tap "**Match Their Vibes**" to sync to the host's playback.
- The app grabs the host's current track + timestamp and commands the listener's connected streaming service to play from that exact position.
- Sync pings run every few seconds to correct drift.
- Listener's playback is slaved to the host — skips, pauses, and track changes propagate to all listeners.

- The listener can exit at any time without disrupting the host.

3.3 "Listening With" Overlay

- While in a session, both the host and listeners see a small, unobtrusive overlay in the corner of the now-playing screen.
- The overlay shows avatars of everyone currently in the session.
- On the host's end: a subtle visual + optional haptic notification when a new listener joins — never interruptive.
- On the listener's end: a real-time view of who else is alongside them.

3.4 The Pulse Acknowledgement

- At any point during a session, either party can trigger a **Pulse** — a gentle synchronized animation that ripples across all connected screens.
- The Pulse is the digital equivalent of making eye contact during a good moment in a song.
- Intentional and manual only — never automatic. No text, no emoji. Just a shared visual beat.

3.5 Friends & Discovery

- Users build a friends list via username search or shareable invite link.
- Friend requests require mutual acceptance.
- The friend list shows who is currently listening (live indicator) and who is offline.
- Tapping a live friend immediately surfaces their profile/stage.

3.6 Vibe History (V2)

- A log of past shared sessions: who you listened with, what was playing, when.
- Functions as a musical scrapbook — the moments you've connected over.

- Private by default; optionally shareable.

4 — PRIVACY MODEL

Music listening is deeply personal. Consent is non-negotiable.

SETTING	DESCRIPTION
Open Door	Friends can drop into your session without a prompt. You'll see a notification they joined, but no approval needed.
Knock First	Friends send a join request; host must accept before sync begins. <i>Default.</i>
Ghost Mode	You are listening privately. No one sees your live status. You can still drop into others' Open Door sessions or send knocks.

- Users can toggle their mode at any time from the now-playing screen.
- Specific friends can be blocked from seeing live status regardless of global setting.

5 — TECHNICAL ARCHITECTURE

5.1 Playback Layer Strategy

Tether operates as a **coordination layer** on top of existing streaming services — not a streaming service itself. It does not serve audio.

Supported platforms (V1): Spotify (via Spotify Connect API), Apple Music (via MusicKit).

How it works:

1. Host connects their streaming service account via OAuth.
2. Host begins playing music in Spotify or Apple Music on any device.
3. Tether polls the host's playback state every ~500ms — capturing `track_id`, `position_ms`, `is_playing`, `timestamp`.
4. When a listener joins, Tether fires a seek command to the listener's streaming service: play `track_id` from `position_ms`.
5. Tether re-syncs every 3–5 seconds to correct drift.

Cross-platform sync: Tracks are matched across services using the **ISRC code** (International Standard Recording Code) — a universal identifier embedded in both platforms' metadata. If a track doesn't exist on the listener's platform, the app surfaces an error and allows passive following.

5.2 Frontend

- **React** (web) / **React Native** (mobile)
- Real-time state via WebSockets — friend presence, playback state, session participants
- Optimistic UI updates for instantaneous feel

5.3 Backend

- **Python** (FastAPI or Django)
- **PostgreSQL** — user profiles, friend relationships, session history, privacy settings
- **Redis** — real-time presence state, playback snapshots, session membership
- **WebSocket server** — live updates pushed to all connected clients

5.4 Key Technical Risk

Tether is dependent on Spotify and Apple Music API access. Spotify in particular has a history of restricting third-party playback control. This is the primary platform risk. Long-term mitigation: build a native playback layer, at the cost of music licensing complexity.

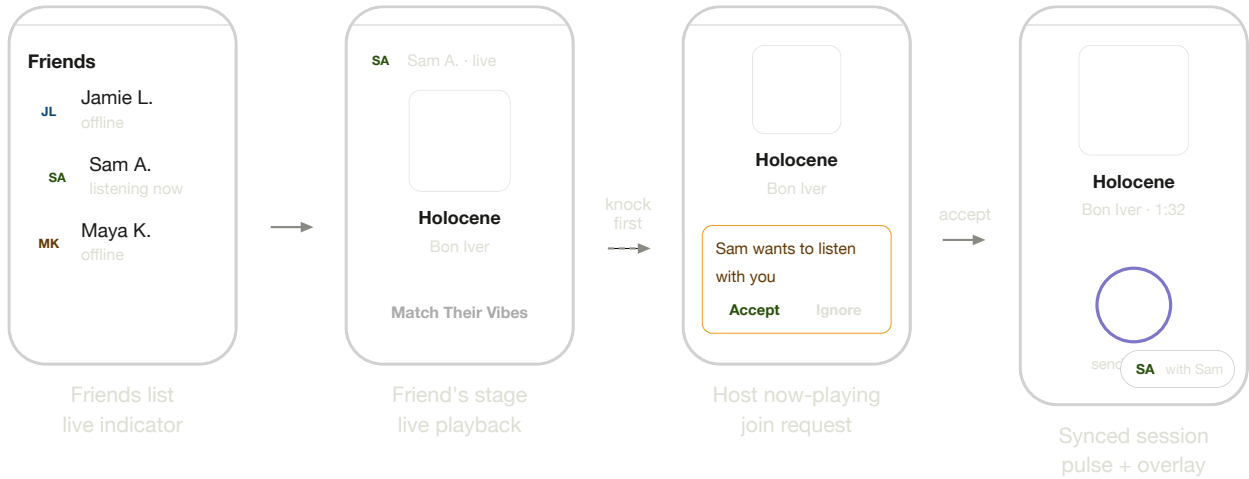
6 — USER FLOWS

WIREFRAME MAP — SCREEN-BY-SCREEN

○ Host ○ Listener ○ Live / active → Conditional → Direct tap

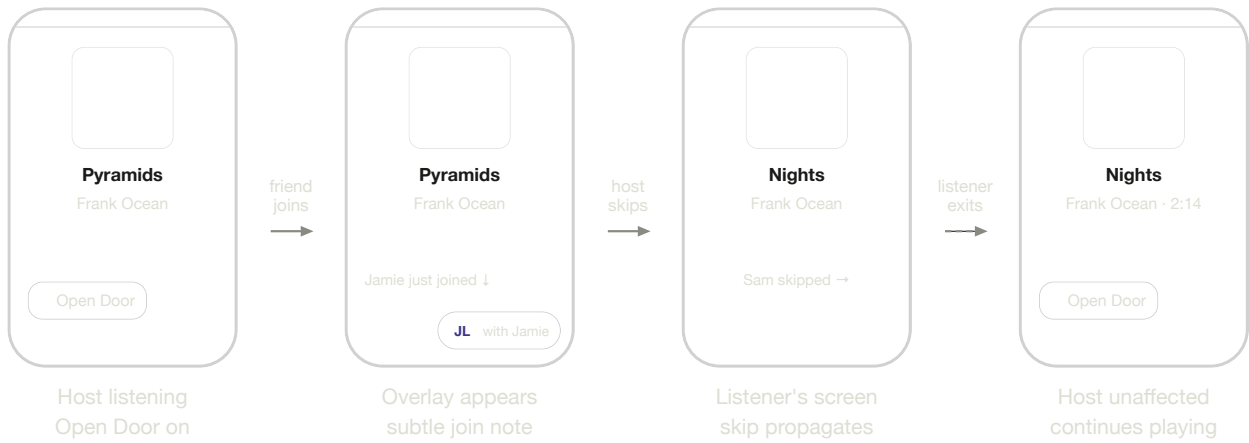
FLOW 1

Joining a friend's session



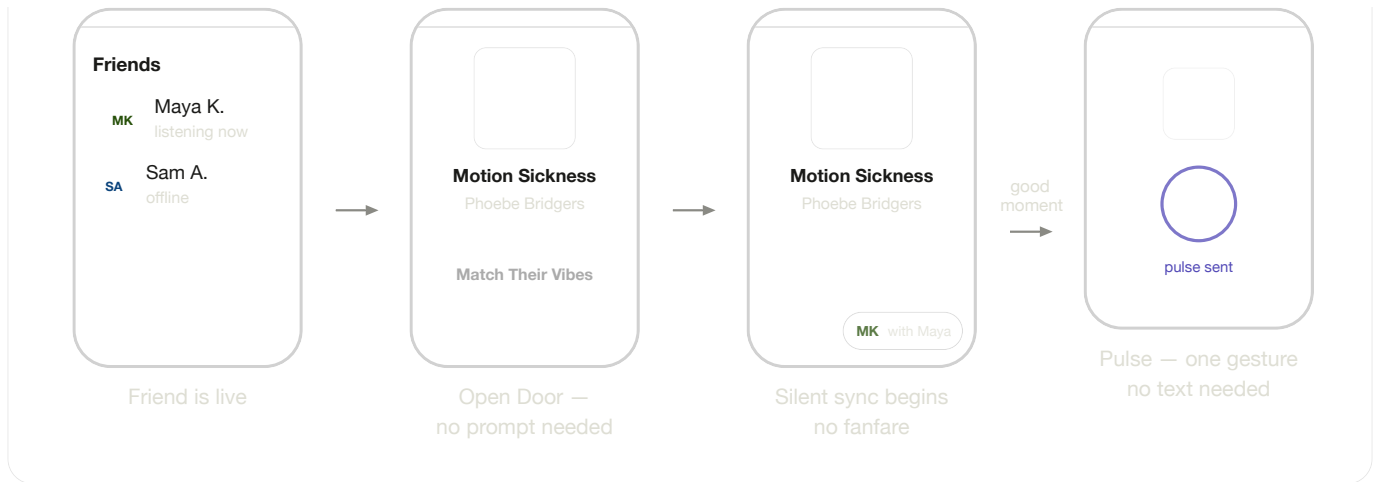
FLOW 2

Hosting — someone drops into your session



FLOW 3

Silent / ambient connection



7 — DESIGN PRINCIPLES

1. **Presence over performance.** The app should feel like being in the same room, not like broadcasting.
2. **The silent experience is as important as the active one.** No feature should force interaction on someone who just wants to be near someone.
3. **Consent is structural.** Privacy controls are not buried in settings — they are a first-class part of the listening UI.
4. **The Pulse is sacred.** It must never become a notification, a reaction carousel, or a metric. It is a single, meaningful gesture.
5. **Invisible sync.** The technical complexity of cross-platform sync should be completely invisible to the user. If it requires a loading screen or an explanation, it has failed.

8 — OUT OF SCOPE (V1)

- Group sessions with more than ~10 concurrent listeners
- Voice or chat built into the app — use FaceTime or a call alongside
- Algorithmic recommendations based on friend listening data

- Public rooms or discovery of strangers' sessions
- Native streaming (music licensing complexity)

9 — OPEN QUESTIONS

What happens when the host pauses?

Does the listener's playback pause too, or do they continue independently? Likely: pause together, with a UI indicator that the host paused — preserving the shared moment.

Cross-platform error state

If the listener uses Spotify and the host plays a track not on Spotify, what's the fallback? Show track info but no audio? Suggest switching platforms?

Session size cap

Is there a cap on concurrent listeners per host? Affects both infrastructure load and the social dynamics of the experience.

Does the listener see the host's queue?

Knowing what's coming next adds to the shared experience but may feel invasive depending on the relationship.

Document maintained by Dylan · Last updated May 2026 · v0.1 Ideation